ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group XX: [names and student IDs]

2026-07-15

Contents

Authorship and contribution statement **2**

Abstract **2**

Reproducible setup **2**

Shared helper functions 3

1 Problem 1. Prediction design and leakage control **5**

1.1 1A. Target and predictors 5

1.2 1B. Split and train-only preprocessing 6

1.3 1C. Training-data exploration 6

1.4 Pairwise correlation among predictors 6

1.5 Response Distribution 8

2 Problem 2. OLS, ridge, and lasso **14**

2.1 2A. OLS baseline 14

2.2 2B. Ridge 14

2.3 2C. Lasso 14

2.4 2D 19

2.5 2E. Perturbation or stability check 19

3 Problem 3. Mathematical mechanisms **20**

3.1 3A. Ridge and conditioning 20

3.2 3B. Lasso optimality and soft thresholding 20

3.3 3C. Connect algebra to evidence 20

4 Problem 4. Elastic net and a neural-feature bridge **20**

4.1 4A. Research question and source map 20

4.2 4B. Elastic net on original predictors 20

4.3 4C. Fixed ReLU features 21

4.4 4D. Locked declaration and final holdout evaluation 21

4.5 4E. Deep-learning connection and limitations 21

5 Problem 5. Report quality and reproducibility **21**

5.1 5A. Reproduction record 21

5.2 5B. Recommendation and limitations 21

5.3 5C. AI verification record 22

Preflight and session information **22**

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in `Group04_AI_Log.csv`, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
Nguyen Thi Yen Nhi	24125071	[Work]	[Check]
Nguyen Khang Thinh	24125104	[Work]	[Check]
Nguyen Van Tinh	24125106	[Work]	[Check]
Nguyen Le Quoc Huy	24125100	[Work]	[Check]

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr", "car")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
```

```
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)
library(car)

group_number <- as.integer(params$group_number)
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)
```

Shared helper functions

```
find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}
```

```r
fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
```

```
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}
```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```
config <- list(
  file = "prostate.csv",
  response = "lpsa",
  excluded = "lpsa"
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}
```

Define the prediction task and justify every exclusion.

1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test
train_df <- analysis_data[train_id, ]

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]


foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
sum(is.na(analysis_data))
```

```
## [1] 0
```

```
save(x_train, y_train, x_test, y_test, analysis_data, train_id, test_id, foldid, file = "data/t
```

Report seeds, dimensions, scaling, folds, and leakage safeguards.

1.3 1C. Training-data exploration

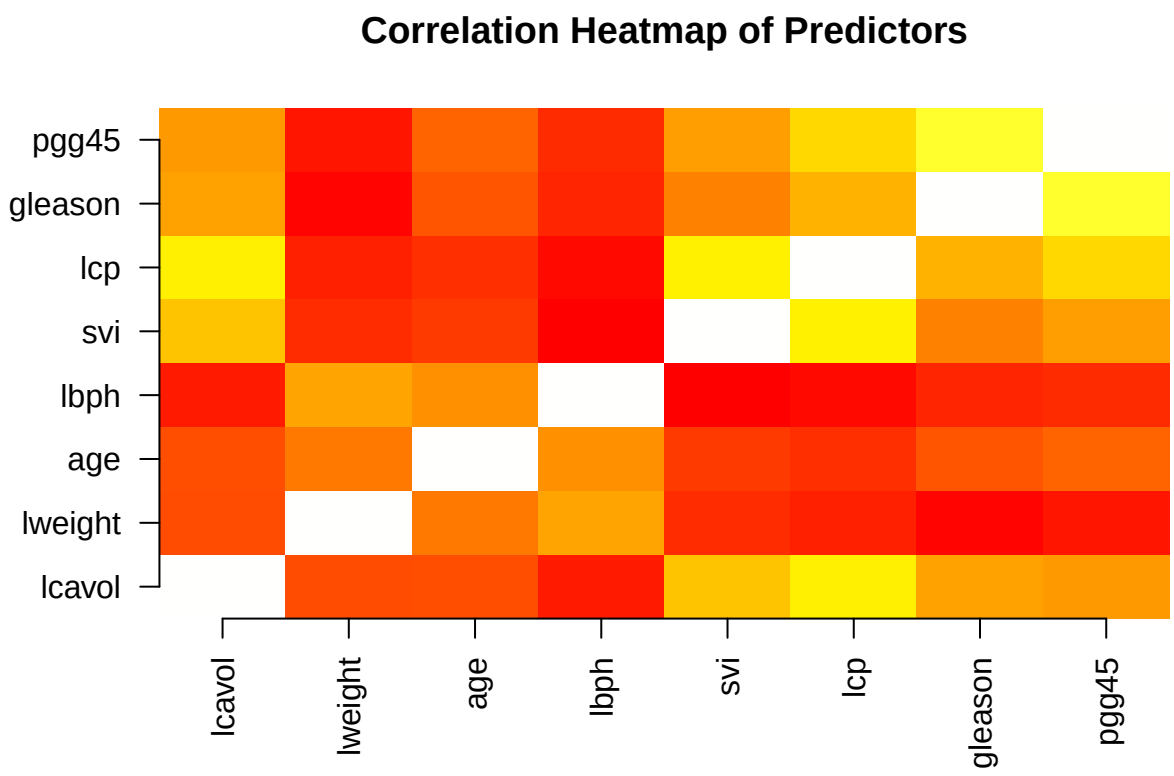
1.4 Pairwise correlation among predictors

```
# Pairwise correlation among predictors
cor_matrix <- cor(
  train_df[, predictors],
  use = "complete.obs"
)

knitr::kable(
  round(cor_matrix, 2),
  caption = "Pairwise correlation matrix among predictors"
)
```

Table 1: Pairwise correlation matrix among predictors

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45
lcavol	1.00	0.17	0.17	0.01	0.55	0.68	0.44	0.41
lweight	0.17	1.00	0.31	0.44	0.07	0.03	-0.06	-0.01
age	0.17	0.31	1.00	0.38	0.11	0.08	0.19	0.24
lbph	0.01	0.44	0.38	1.00	-0.07	-0.04	0.05	0.06
svi	0.55	0.07	0.11	-0.07	1.00	0.69	0.33	0.43
lcp	0.68	0.03	0.08	-0.04	0.69	1.00	0.49	0.61
gleason	0.44	-0.06	0.19	0.05	0.33	0.49	1.00	0.78
pgg45	0.41	-0.01	0.24	0.06	0.43	0.61	0.78	1.00

```r
# Correlation heatmap
cor_matrix <- cor(train_df[, predictors], use = "complete.obs")

image(1:ncol(cor_matrix), 1:ncol(cor_matrix), cor_matrix,
      axes = FALSE, xlab = "", ylab = "",
      col = heat.colors(1000), main = "Correlation Heatmap of Predictors")
axis(1, at = 1:ncol(cor_matrix), labels = colnames(cor_matrix), las = 2)
axis(2, at = 1:ncol(cor_matrix), labels = colnames(cor_matrix), las = 1)
```

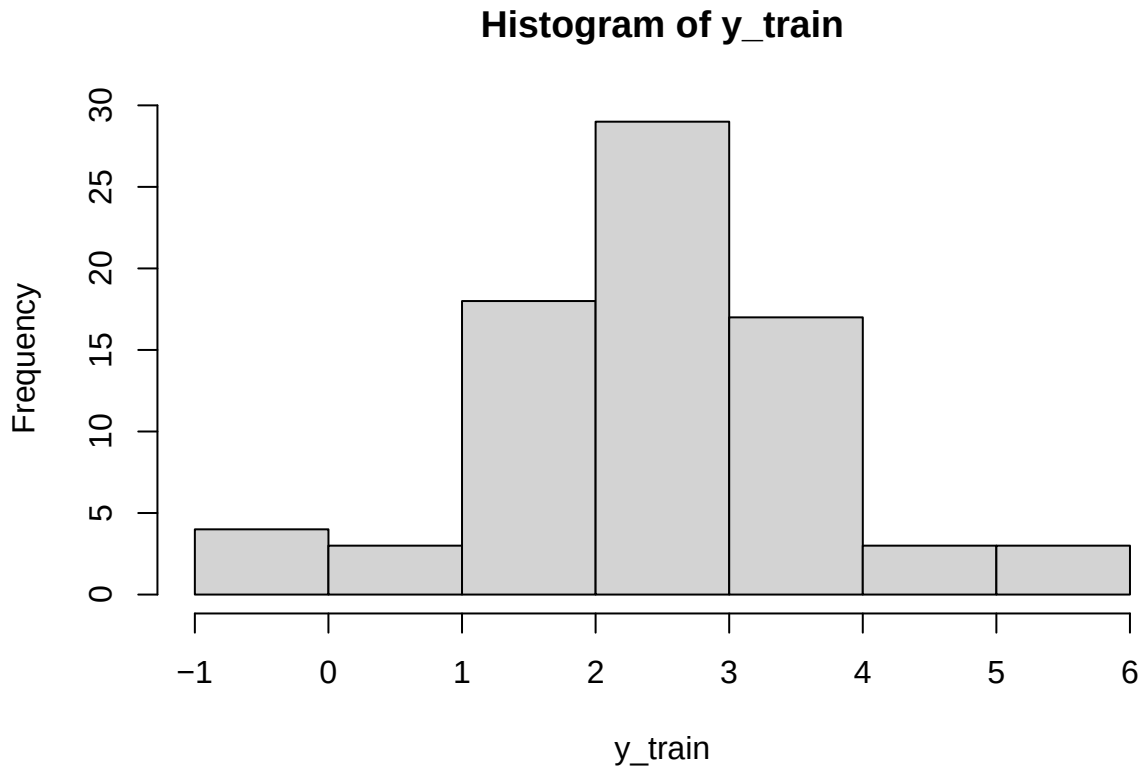


The pairwise correlation matrix was examined to identify potential relationships among predictors.

Most predictors showed weak to moderate correlations, suggesting limited redundancy between features. However, some stronger positive correlations were observed, particularly between gleason and pgg45 (r = 0.78), lcavol and lcp (r = 0.68), and svi and lcp (r = 0.69). These relationships indicate that some predictors may share related clinical information. Therefore, Variance Inflation Factor (VIF) analysis was further performed to assess the overall multicollinearity effect in the regression model.

1.5 Response Distribution

```
hist(y_train)
```



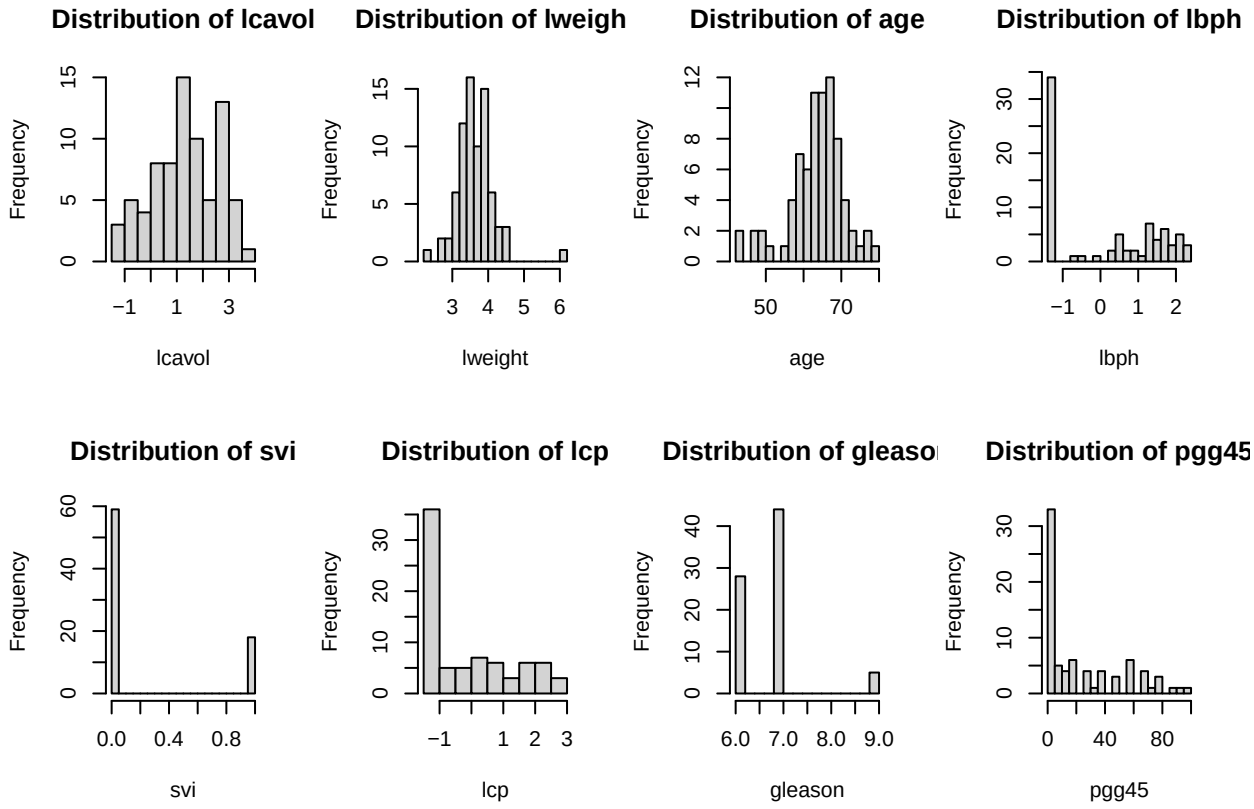

```
summary(y_train)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.4308  1.7664  2.5688  2.4757  3.2753  5.5829
```

The distribution of the response variable (lpsa) was examined using a histogram and summary statistics. The response values range from -0.43 to 5.58, with a median of 2.57 and a mean of 2.48. The mean and median are relatively close, indicating that the distribution is approximately symmetric with no strong skewness. Although some observations appear at the lower and higher ends of the range, no severe outliers are evident from the overall distribution. Therefore, no transformation was applied to the response variable before model training.

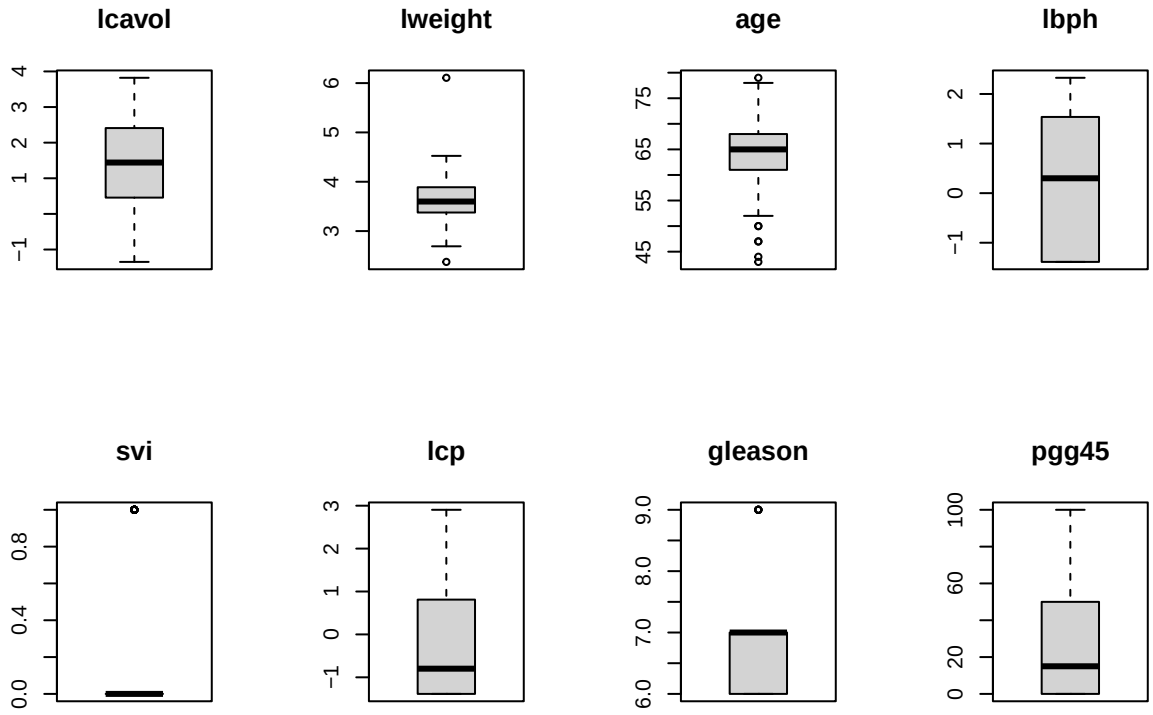
1.5.1 Predictor Histogram & Boxplot

```
## Histogram
par(mfrow = c(2,4))

for (var in predictors) {
  hist(
    train_df[[var]],
    main = paste("Distribution of", var),
    xlab = var,
    breaks = 15
  )
}
```




```
par(mfrow = c(1,1))

## Boxplot
par(mfrow = c(2,4))
for (var in predictors) {
  boxplot(
    train_df[[var]],
    main = var
```

```
  )
}
```




```
par(mfrow = c(1,1))
```

The histograms show that most predictors have different distribution patterns. lcavol, lweight, and age are approximately symmetric, while lbph, lcp, and pgg45 exhibit right-skewed distributions with longer tails toward higher values. svi and gleason are discrete variables with highly concentrated values. These distribution characteristics are considered in later regression analysis and model diagnostics.

Upon inspecting the visualizations, a potential influential observation is evident in the lweight vs lpsa plot. There is a single data point located at the far right with an exceptionally high lweight (exceeding 6.0) but a relatively low lpsa (approximately 2.0).

1.5.2 Correlation between predictors and response

```
cor_response <- cor(
  train_df[, predictors],
  train_df$lpsa
)
```

```r
knitr::kable(
  round(cor_response, 2),
  caption = "Correlation between predictors and lpsa"
)
```

Table 2: Correlation between predictors and lpsa

lcavol	0.77
lweight	0.30
age	0.14
lbph	0.14
svi	0.57
lcp	0.59
gleason	0.39
pgg45	0.41

The correlation analysis indicates that all predictors have positive correlations with lpsa. lcavol shows the strongest relationship (r = 0.77), followed by lcp (r = 0.59) and svi (r = 0.57). Variables such as pgg45, gleason, and lweight have moderate correlations, while age and lbph show weak associations. Therefore, lcavol, lcp, and svi are likely to be important predictors in the multiple linear regression model.

1.5.3 Check multicollinearity using VIF

```r
vif_model <- lm(
  lpsa ~ .,
  data = train_df[, c("lpsa", predictors)]
)

vif_values <- vif(vif_model)

knitr::kable(
  round(vif_values, 2),
  caption = "Variance Inflation Factor (VIF)"
)
```

Table 3: Variance Inflation Factor (VIF)

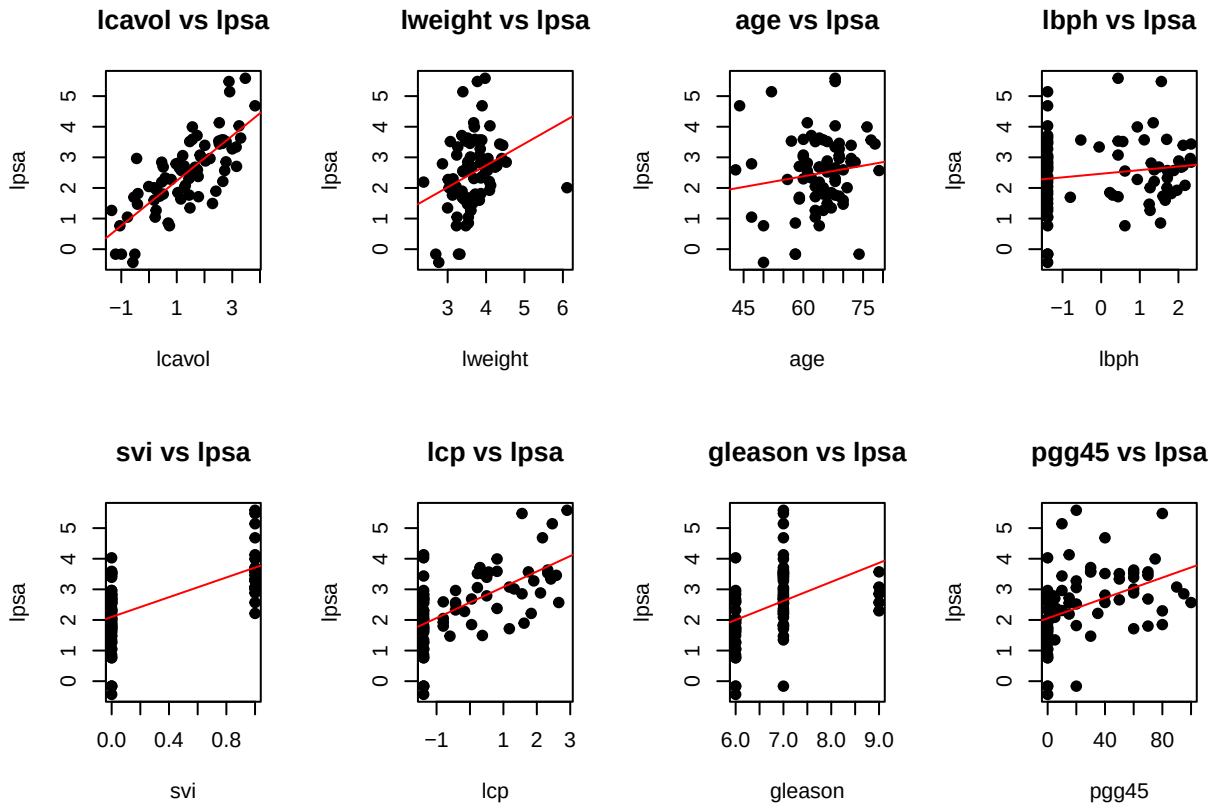
	x
lcavol	2.18
lweight	1.38
age	1.32
lbph	1.40

	x
svi	1.97
lcp	3.29
gleason	2.79
pgg45	3.35

There is no significant multicollinearity present in this model. Because all VIF values are safely below 5, the independent variables are not highly correlated with one another. The regression model's coefficient estimates are mathematically stable, and no variables need to be dropped or transformed for multicollinearity reasons.

1.5.4 Predictor-response plots

```
par(mfrow = c(2,4))

for (var in predictors) {
  plot(
    train_df[[var]],
    train_df$lpsa,
    xlab = var,
    ylab = "lpsa",
    main = paste(var, "vs lpsa"),
    pch = 19
  )

  abline(
    lm(train_df$lpsa ~ train_df[[var]]),
    col = "red"
  )
}
```




```
par(mfrow = c(1,1))
```

The provided scatter plots visualize the marginal relationships between the eight candidate predictors and the target variable, lpsa.

Strong Trends: lcavol demonstrates the clearest and most distinct positive linear correlation with lpsa, suggesting it will be a strong predictor in the model.

Categorical Patterns: Variables such as svi (binary) and gleason (ordinal) display expected clustered structures, with higher categories generally corresponding to higher lpsa values.

Weak Trends: age and lbph exhibit highly dispersed data clouds with very flat regression lines, indicating weak independent linear relationships with the target.

1.5.4.1 Question Given the varying individual predictive strengths of these features and the moderate structural correlations (e.g., between pgg45 and lcp), will an L_1 penalty (Lasso) completely eliminate the structurally weaker predictors like age to produce a strictly sparse model, or will an L_2 penalty (Ridge) prove more stable against the identified high-leverage anomaly in lweight?

2 Problem 2. OLS, ridge, and lasso

2.1 2A. OLS baseline

```
# Fit OLS on x_train and y_train. Do not access y_test here.
```

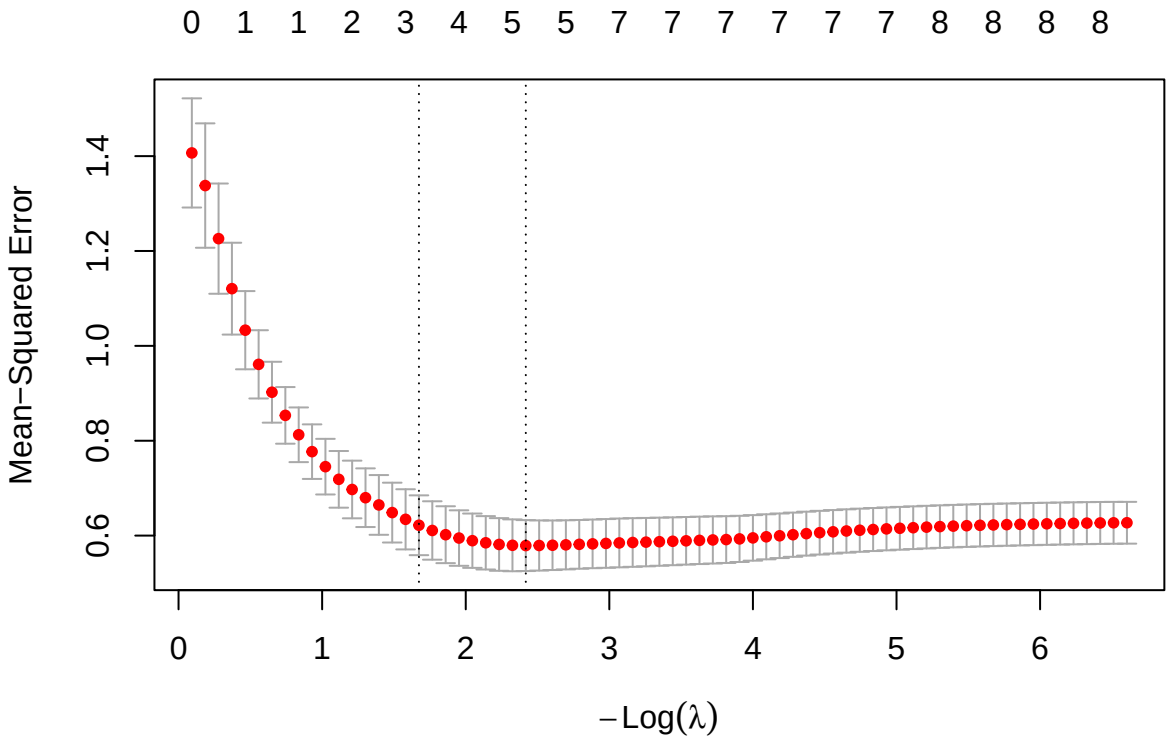
2.2 2B. Ridge

```
# Use fit_cv_glmnet(..., alpha = 0, foldid = foldid).
# Report lambda.min, lambda.1se, paths, coefficients, and CV evidence.
```

2.3 2C. Lasso

2.3.1 Model fitting and cross-validation

```
lasso_cv <- fit_cv_glmnet(
    x = x_train,
    y = y_train,
    alpha = 1,
    foldid = foldid
)
plot(lasso_cv)
```



Based on the 5-fold cross-validation curve generated for the Lasso regression model ($\alpha = 1$), we observe the relationship between the tuning parameter λ and the Mean-Squared Error (MSE). The top axis indicates the number of active (non-zero) predictors retained in the model at each level of penalization.

- The Cross-Validation Curve: The red dots represent the average CV-MSE across the 5 folds for each λ value, with the gray error bars denoting one standard error above and below the mean. As $-\log(\lambda)$ increases (moving from left to right, meaning the penalty weakens), the MSE initially drops, reaches a minimum, and then slightly stabilizes as the model approaches the unpenalized OLS limit.
- Optimal λ (λ_{min}): The right vertical dashed line marks `lambda.min`, the value that strictly minimizes the cross-validation MSE. At this optimal point for prediction accuracy ($-\log(\lambda) \approx 2.4$), the Lasso penalty retains 5 non-zero predictors*.
- Parsimonious λ (λ_{1se}): The left vertical dashed line marks `lambda.1se`, selected by the one-standard-error rule. This value applies a stronger penalty ($-\log(\lambda) \approx 1.67$) to produce a sparser model. It retains only 3 non-zero predictors while keeping the error within one standard deviation of the absolute minimum MSE, offering a robust trade-off between predictive power and model simplicity.

2.3.2 Analysis of λ_{min}

```
lambda_min <- lasso_cv$lambda.min
coef(lasso_cv, s="lambda.min")
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##             lambda.min
## (Intercept) 2.47569468
## lcavol      0.67611312
## lweight     0.12774355
## age         .
## lbph        0.03085712
## svi         0.20454921
## lcp         .
## gleason     .
## pgg45       0.03234366
```

```
count_nonzero(lasso_cv, s="lambda.min")
```

```
## [1] 5
```

The optimal tuning parameter selected by five-fold cross-validation is $\lambda_{\text{min}} = 0.0891$. This value minimizes the cross-validation Mean Squared Error (CV-MSE), providing the best predictive performance among all candidate penalty values.

At this level of regularization, the Lasso model retains five non-zero predictors: lcavol, lweight, lbph, svi, and pgg45. The remaining predictors (age, lcp, and gleason) are shrunk exactly to zero, demonstrating Lasso's ability to perform automatic variable selection while maintaining predictive accuracy.

The retained predictors suggest that tumor volume (lcavol) is the most influential variable, having the largest estimated coefficient (0.6761). Variables related to body weight (lweight), benign prostatic hyperplasia (lbph), seminal vesicle invasion (svi), and the percentage of Gleason grades 4 or 5 (pgg45) also contribute to predicting PSA levels, although with relatively smaller effects.

Overall, λ_{min} produces the model with the lowest cross-validation error while preserving a moderate number of predictors, offering a balance between prediction accuracy and model complexity.

```
cv_mse_min <- lasso_cv$cvm[
    lasso_cv$lambda==
    lasso_cv$lambda.min
]
cv_mse_min
```

```
## [1] 0.5789934
```

The minimum five-fold cross-validation Mean Squared Error (CV-MSE) achieved by the Lasso model is **0.579**. This value corresponds to $\lambda_{\text{min}} = 0.0891$ and represents the estimated prediction error obtained during model tuning using the training data.

A lower CV-MSE indicates better expected predictive performance on unseen data. Since λ_{min} is selected to minimize this metric, it is the tuning parameter that prioritizes prediction accuracy over model simplicity.

2.3.3 Analysis of λ_{1se}

```
lambda_1se <- lasso_cv$lambda.1se
coef(lasso_cv, s="lambda.1se")
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##             lambda.1se
## (Intercept) 2.47569468
## lcavol      0.63661195
## lweight     0.05343261
## age         .
## lbph        .
## svi         0.14411468
## lcp         .
## gleason     .
## pgg45       .
```

```
count_nonzero(lasso_cv, s="lambda.1se")
```

```
## [1] 3
```

The one-standard-error rule selects $\lambda_{\text{1se}} = rround(lambda_1 se, 4)$, which applies a stronger regularization penalty than λ_{min}. The corresponding five-fold cross-validation Mean Squared Error (CV-MSE) is r round(cv_mse_1se, 4), remaining within one standard error of the minimum while producing a simpler model.

Under λ_{1se}, the Lasso model retains only three non-zero predictors (lcavol, lweight, and svi). Compared with λ_{min}, the variables lbph and pgg45 are removed, resulting in a more parsimonious and interpretable model with only a small loss in predictive performance.

```
cv_mse_1se <- lasso_cv$cvm[
    lasso_cv$lambda==
    lasso_cv$lambda.1se
]
cv_mse_1se
```

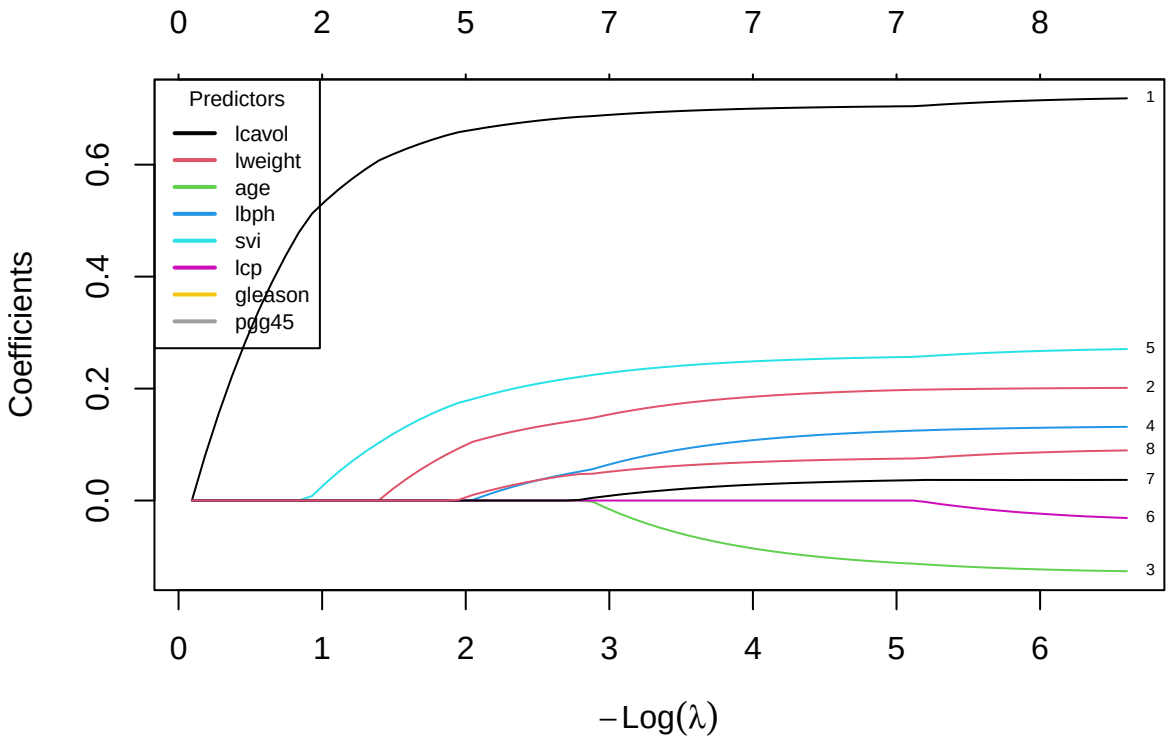
```
## [1] 0.6217887
```

The one-standard-error rule selects $\lambda_{\text{1se}} = rround(lambda_1 se, 4)$, resulting in a five-fold cross-validation Mean Squared Error (CV-MSE) of r round(cv_mse_1se, 4). Although this error is slightly higher than that of λ_{min}, it remains within one standard error of the minimum while providing a more conservative model.

Under λ_{1se}, the Lasso model retains only three non-zero predictors (lcavol, lweight, and svi). Compared with λ_{min}, the predictors lbph and pgg45 are removed, resulting in a simpler and more interpretable model with only a modest reduction in predictive performance.

2.3.4 Coefficient Path

```
lasso_fit <- glmnet(
    x_train,
    y_train,
    alpha=1,
    standardize=FALSE,

)

plot(lasso_fit, xvar = "lambda", label = TRUE)

legend("topleft",
       legend = rownames(lasso_fit$beta),
       col = 1:nrow(lasso_fit$beta),
       lty = 1,
       cex = 0.7,
       lwd = 2,
       title = "Predictors")
```



Overall, the coefficient path shows that as $-\log(\lambda)$ increases (equivalently, as λ decreases), the regularization penalty becomes weaker, allowing more predictors to enter the model and their coefficients to increase in magnitude. Under strong regularization, many coefficients are shrunk exactly to zero, while weaker regularization gradually recovers the full model.

Among the predictors, **lcavol** enters the model first and consistently has the largest coefficient, indicating that it is the most influential predictor of **lpsa**. **svi** and **lweight** also become active relatively early and maintain positive coefficients throughout the path. In contrast, **age, lbph, lcp, gleason,** and **pgg45** enter only when the penalty becomes sufficiently small, suggesting that their contributions are comparatively weaker and less stable.

2.4 2D

```
# Build a comparison from training/CV quantities only.
```

Core model nominated before holdout evaluation: [method and reason]

2.5 2E. Perturbation or stability check

Prediction before running the check: [prediction]

```
# Implement one allowed training-only stability check.
```

Observed result and explanation: [result]

3 Problem 3. Mathematical mechanisms

3.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

```
# Use x_train and the fitted ridge lambda.min.
# safe_condition_numbers(x_train, lambda = ...)
```

3.2 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

3.3 3C. Connect algebra to evidence

Connect one specific numerical result to Problem 3A or 3B.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map

Research direction: [weight decay / sparse weights / pruning / dropout]

Claim	Exact primary source and locator	What it establishes	What it does not establish
[Claim]	[Citation, section/page/equation]	[Support]	[Boundary]

4.2 4B. Elastic net on original predictors

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)

# Fit one cv.glmnet object per alpha with the same foldid.
# Select alpha and lambda using training CV only.
```

4.3 4C. Fixed ReLU features

```r
relu <- function(z) pmax(z, 0)
M <- 200L

# TODO:
# 1. Generate A and bias once from seeds$features using the stated distributions.
# 2. Apply the same A and bias to x_train and x_test.
# 3. Fit a scaler to H_train only and apply it to H_test.
# 4. Fit ridge, lasso, and elastic net on H_train with the shared foldid.
```

State which weights are fixed, which are learned, and how active features are counted.

4.4 4D. Locked declaration and final holdout evaluation

> **Locked before viewing `y_test`:** [preprocessing, candidate specifications, selected tuning values, feature seed, nominated deployment model, metrics]

```r
# This must be the first analysis chunk that uses y_test for evaluation.
# Generate all fixed candidate predictions and one RMSE/MAE table here.
```

State whether the holdout evidence supports the predeclared choice. Do not retune.

4.5 4E. Deep-learning connection and limitations

Distinguish penalty, weight decay, output-weight sparsity, pruning, dropout, and a trained deep network. Cite the sources from 4A and discuss at least two limitations.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

```
1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.
```

5.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Vietnamese_Vietnam.utf8  LC_CTYPE=Vietnamese_Vietnam.utf8
## [3] LC_MONETARY=Vietnamese_Vietnam.utf8 LC_NUMERIC=C
## [5] LC_TIME=Vietnamese_Vietnam.utf8
##
## time zone: Asia/Saigon
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] car_3.1-5       carData_3.0-6   knitr_1.51      broom_1.0.13
##  [5] glmnet_5.0      Matrix_1.7-5    lubridate_1.9.5 forcats_1.0.1
```

```
##  [9] stringr_1.6.0   dplyr_1.2.1     purrr_1.2.2     readr_2.2.0
## [13] tidyr_1.3.2     tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     shape_1.4.6.1      stringi_1.8.7      lattice_0.22-9
##  [5] hms_1.1.4          digest_0.6.39      magrittr_2.0.5     timechange_0.4.0
##  [9] evaluate_1.0.5     grid_4.6.1         RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0      foreach_1.5.2      backports_1.5.1    Formula_1.2-5
## [17] survival_3.8-6     scales_1.4.0       codetools_0.2-20   abind_1.4-8
## [21] cli_3.6.6          rlang_1.3.0        splines_4.6.1      withr_3.0.3
## [25] yaml_2.3.12        otel_0.2.0         tools_4.6.1        tzdb_0.5.0
## [29] vctrs_0.7.3        R6_2.6.1           lifecycle_1.0.5    pkgconfig_2.0.3
## [33] pillar_1.11.1      gtable_0.3.6       glue_1.8.1         Rcpp_1.1.2
## [37] xfun_0.60          tidyselect_1.2.1   rstudioapi_0.19.0  farver_2.1.2
## [41] htmltools_0.5.9    rmarkdown_2.31     compiler_4.6.1     S7_0.2.2
```